

PR_12.1 Dani Gayol Rodríguez

PR_12.1 Dani Gayol Rodríguez.....	1
Apartado A.....	1
1.) Crea en tu servidor MySQL una base de datos llamada empleados.	3
2.) Dentro de la base de datos anterior crea las tablas con los campos que se ven abajo y relaciones adecuadas entre las tablas:	4
3.) Con las sentencias SQL adecuadas añade a cada tabla los registros siguientes: ..	6
4.) Realiza las siguientes consultas mostrando también su salida por pantalla.	9
1. Seleccionar el nº de empleado, salario, comisión, nº de departamento y fecha de la tabla EMP.	9
2. Seleccionar todas las columnas de la tabla DEPT.	10
3. Seleccionar los nombres y los empleos de todos los empleados, ordenados por empleo.	10
4. Seleccionar los empleos que hay en cada departamento, ordenados por departamento.....	11
5. Seleccionar los distintos departamentos que existen en la tabla EMP.	11
6. Calcular el salario anual a percibir por cada empleado.	12
7. Mostrar el nombre del empleado y una columna que contenga el salario multiplicado por la comisión cuya cabecera sea “BONO”.	12
8. Seleccionar aquellos empleados que sean "SALESMAN".	13
9. Seleccionar aquellos empleados que no trabajen en el departamento 30.	13
10. Seleccionar el nombre de aquellos empleados que ganen más de 2000.	13
11. Seleccionar aquellos empleados que hayan entrado antes del 1/1/82	14
12. Mostrar el nombre del empleado y su fecha de alta en la empresa de los empleados que son “ANALISTA”.	14
13. Seleccionar los empleados cuyo salario sea superior al de “ADAMS”.	14
14. Seleccionar los empleados que trabajan en el mismo departamento que "CLARK"	15
15. Encontrar a los empleados cuyo jefe es “BLAKE”.	15
16. Seleccionar el nombre de los vendedores que ganen más de 1500.	16

17. Seleccionar aquellos empleados que tienen comisión.	16
18. Seleccionar aquellos que se llamen "SMITH", "ALLEN" o "SCOTT ".....	16
19. Seleccionar aquellos que no se llamen "SMITH", "ALLEN" o "SCOTT ".....	17
20. Seleccionar los empleados que trabajen en "CHICAGO".	17
21. Seleccionar aquellos empleados que trabajen en el departamento 10 o en el 20.	17

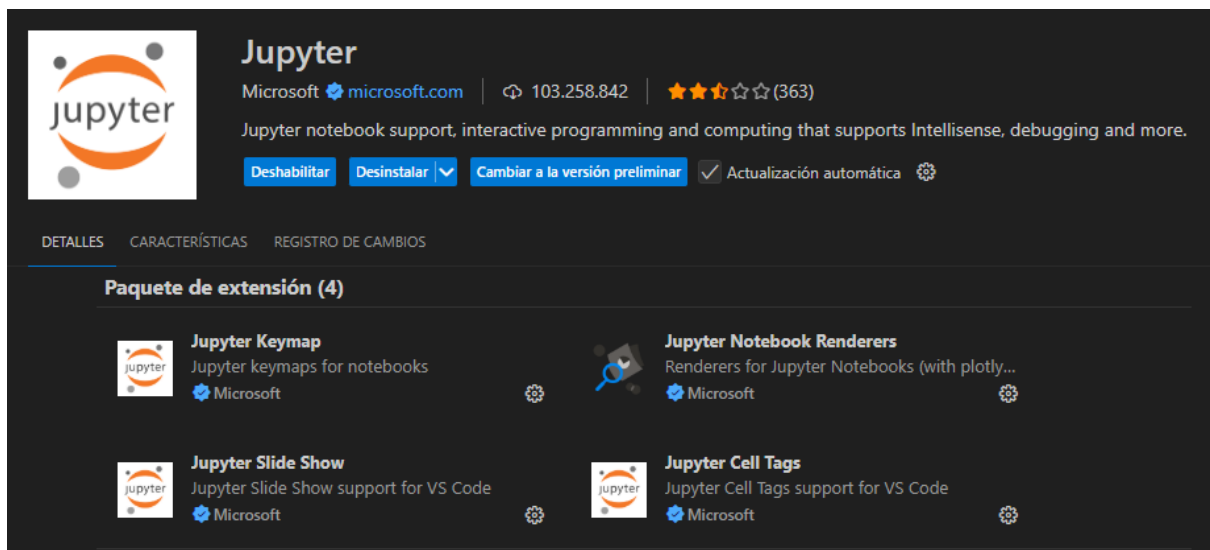
Apartado A

Instalar todas las extensiones necesarias de Visual Studio Code:



Data Wrangler
Microsoft  microsoft.com | 1.927.551 | ★★★★★ (73)
Data viewing, cleaning and preparation for tabular datasets
[Deshabilitar] [Desinstalar] [Actualización automática]

DETALLES CARACTERÍSTICAS REGISTRO DE CAMBIOS

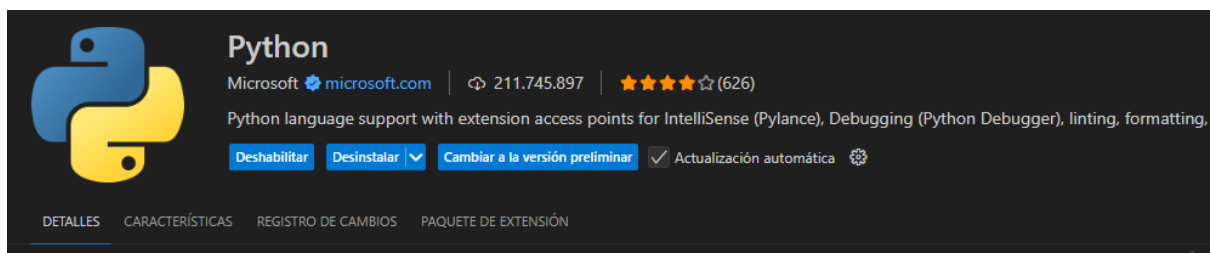


Jupyter
Microsoft  microsoft.com | 103.258.842 | ★★★★★ (363)
Jupyter notebook support, interactive programming and computing that supports IntelliSense, debugging and more.
[Deshabilitar] [Desinstalar] [Cambiar a la versión preliminar] [Actualización automática]

DETALLES CARACTERÍSTICAS REGISTRO DE CAMBIOS

Paquete de extensión (4)

Extension Name	Description	Provider
Jupyter Keymap	Jupyter keymaps for notebooks	Microsoft
Jupyter Notebook Renderers	Renderers for Jupyter Notebooks (with plotly...)	Microsoft
Jupyter Slide Show	Jupyter Slide Show support for VS Code	Microsoft
Jupyter Cell Tags	Jupyter Cell Tags support for VS Code	Microsoft



Python
Microsoft  microsoft.com | 211.745.897 | ★★★★★ (626)
Python language support with extension access points for IntelliSense (Pylance), Debugging (Python Debugger), linting, formatting.
[Deshabilitar] [Desinstalar] [Cambiar a la versión preliminar] [Actualización automática]

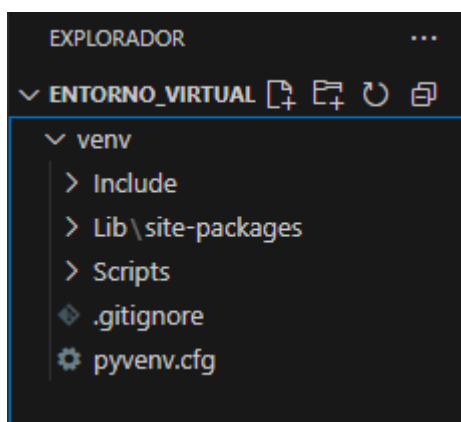
DETALLES CARACTERÍSTICAS REGISTRO DE CAMBIOS PAQUETE DE EXTENSIÓN

Crear un entorno virtual:

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
● PS C:\Users\Mañana\Documents\entorno_virtual> python -m venv venv
● PS C:\Users\Mañana\Documents\entorno_virtual> venv\Scripts\activate
● (venv) PS C:\Users\Mañana\Documents\entorno_virtual> pip install mysql-connector-python pandas
Collecting mysql-connector-python
  Downloading mysql_connector_python-9.6.0-cp313-cp313-win_amd64.whl.metadata (11 kB)
Collecting pandas
  Downloading pandas-3.0.2-cp313-cp313-win_amd64.whl.metadata (19 kB)
Collecting numpy>=1.26.0 (from pandas)
  Downloading numpy-2.4.4-cp313-cp313-win_amd64.whl.metadata (6.6 kB)
Collecting python-dateutil>=2.8.2 (from pandas)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting tzdata (from pandas)
  Downloading tzdata-2026.1-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Downloading mysql_connector_python-9.6.0-cp313-cp313-win_amd64.whl (16.5 MB)
  16.5/16.5 MB 541.0 kB/s 0:00:41
Downloading pandas-3.0.2-cp313-cp313-win_amd64.whl (9.7 MB)
  9.7/9.7 MB 8.4 MB/s 0:00:01
Downloading numpy-2.4.4-cp313-cp313-win_amd64.whl (12.3 MB)
  12.3/12.3 MB 6.5 MB/s 0:00:01
Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading tzdata-2026.1-py2.py3-none-any.whl (348 kB)
Installing collected packages: tzdata, six, numpy, mysql-connector-python, python-dateutil, pandas
Successfully installed mysql-connector-python-9.6.0 numpy-2.4.4 pandas-3.0.2 python-dateutil-2.9.0.post0 six-1.17.0 tzdata-2026.1

[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
○ (venv) PS C:\Users\Mañana\Documents\entorno_virtual>
```

```
● (venv) PS C:\Users\Mañana\Documents\entorno_virtual> pip install mysql-connector-python ipykernel
Requirement already satisfied: mysql-connector-python in c:\users\manana\documents\entorno_virtual\venv\lib\site-packages (9.6.0)
Collecting ipykernel
  Downloading ipykernel-7.2.0-py3-none-any.whl.metadata (4.5 kB)
Collecting comm>=0.1.1 (from ipykernel)
  Downloading comm-0.2.3-py3-none-any.whl.metadata (3.7 kB)
```



Crear una base de datos Mysql en Aws Academy RDS:

Bases de datos (1)

Recursos del grupo

Modificar Acciones Crear base de datos

Filtrar por bases de datos

Identificador de base de datos	Estado	Rol	Motor	Orden de implementa...	Región ...	Tamaño	Recomendaciones	CPU
database-pr-12-1	Dispon...	Instancia	MySQL Co...	SECOND	us-east-1a	db.t3.micro		6.07%

1.) Crea en tu servidor MySQL una base de datos llamada empleados.

Primero, nos tenemos que conectar:

1. Conexión a MySQL

```
import mysql.connector

conexion = mysql.connector.connect(
    host="database-pr-12-1.chg8ik6e82oj.us-east-1.rds.amazonaws.com",
    user="admin",
    password="favomofe57_N"
)

cursor = conexion.cursor()
print("Conectado correctamente")
```

[4] ✓ 0.7s

... Conectado correctamente

Ahora, ya podemos crear la base de datos llamada “empleados”:

2. Crear base de datos

```
cursor.execute("CREATE DATABASE IF NOT EXISTS empleados")
cursor.execute("USE empleados")

print("Base de datos creada correctamente")
```

[6] ✓ 0.1s

... Base de datos creada correctamente

2.) Dentro de la base de datos anterior crea las tablas con los campos que se ven abajo y relaciones adecuadas entre las tablas:

3. Crear tablas dentro de la base de datos

Tabla DEPT

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS DEPT (
    DEPTNO INT PRIMARY KEY,
    DNAME VARCHAR(50),
    LOC VARCHAR(50)
)
""")

print("Tabla DEPT creada correctamente")
```

[9] ✓ 0.1s

... Tabla DEPT creada correctamente

Tabla SALGRADE

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS SALGRADE (
    GRADE INT PRIMARY KEY,
    LOSAL INT,
    HISAL INT
)
""")

print("Tabla SALGRADE creada correctamente")
```

[10] ✓ 0.1s

... Tabla SALGRADE creada correctamente

Tabla EMP

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS EMP (
    ... ENO INT PRIMARY KEY,
    ... ENAME VARCHAR(50),
    ... JOB VARCHAR(50),
    ... MGR INT,
    ... HIREDATE DATE,
    ... SAL FLOAT,
    ... COMM FLOAT,
    ... DEPTNO INT,
    ... FOREIGN KEY (DEPTNO) REFERENCES DEPT(DEPTNO)
)
""")

print("Tabla EMP creada correctamente")
```

[11] ✓ 0.1s

... Tabla EMP creada correctamente

Comprobar que se crearon las tablas correctamente

```
cursor.execute("SHOW TABLES")
for tabla in cursor:
    print(tabla)
```

[62] ✓ 0.1s

... ('DEPT',)
('EMP',)
('SALGRADE',)

3.) Con las sentencias SQL adecuadas añade a cada tabla los registros siguientes:

4. Insertar a cada tabla los siguientes registros

Valores tabla DEPT

```
cursor.execute("INSERT INTO DEPT VALUES (10, 'ACCOUNTING', 'NEW YORK')")
cursor.execute("INSERT INTO DEPT VALUES (20, 'RESEARCH', 'DALLAS')")
cursor.execute("INSERT INTO DEPT VALUES (30, 'SALES', 'CHICAGO')")
cursor.execute("INSERT INTO DEPT VALUES (40, 'OPERATIONS', 'BOSTON')")

print("Valores insertados correctamente en la tabla DEPT")
```

[63] ✓ 0.3s

... Valores insertados correctamente en la tabla DEPT

Valores tabla SALGRADE

```
cursor.execute("INSERT INTO SALGRADE VALUES (1, 700, 1200)")
cursor.execute("INSERT INTO SALGRADE VALUES (2, 1201, 1400)")
cursor.execute("INSERT INTO SALGRADE VALUES (3, 1401, 2000)")
cursor.execute("INSERT INTO SALGRADE VALUES (4, 2001, 3000)")
cursor.execute("INSERT INTO SALGRADE VALUES (5, 3001, 9999)")

print("Valores insertados correctamente en la tabla DEPT")
```

[64] ✓ 0.4s

... Valores insertados correctamente en la tabla DEPT

Valores tabla EMP

```
cursor.execute("""
INSERT INTO EMP VALUES
(7369, 'SMITH', 'CLERK', 7902, '1980-12-17', 800, NULL, 20)
""")

cursor.execute("""
INSERT INTO EMP VALUES
(7499, 'ALLEN', 'SALESMAN', 7698, '1981-02-20', 1600, 300, 30)
""")

cursor.execute("""
INSERT INTO EMP VALUES
(7521, 'WARD', 'SALESMAN', 7698, '1981-02-22', 1250, 500, 30)
""")

cursor.execute("""
INSERT INTO EMP VALUES
(7566, 'JONES', 'MANAGER', 7839, '1981-04-02', 2975, NULL, 20)
""")
```

Comprobar que se insertaron los valores correctamente

```
print("Valores tabla DEPT:\n")
cursor.execute("SELECT * FROM DEPT")
for fila in cursor:
    print(fila)

print("\n\nValores tabla SALGRADE:\n")
cursor.execute("SELECT * FROM SALGRADE")
for fila in cursor:
    print(fila)

print("\n\nValores tabla EMP:\n")
cursor.execute("SELECT * FROM EMP")
for fila in cursor:
    print(fila)
```

[66] ✓ 0.2s

... Valores tabla DEPT:

```
(10, 'ACCOUNTING', 'NEW YORK')
(20, 'RESEARCH', 'DALLAS')
(30, 'SALES', 'CHICAGO')
(40, 'OPERATIONS', 'BOSTON')
```

4.) Realiza las siguientes consultas mostrando también su salida por pantalla.

5. Realizar las siguientes consultas

Consultas con salida por pantalla

```
def consultar(sql):
    cursor.execute(sql)
    resultados = cursor.fetchall()
    for fila in resultados:
        print(fila)
```

[43] ✓ 0.0s

1. Seleccionar el nº de empleado, salario, comisión, nº de departamento y fecha de la tabla EMP.

```
1. Seleccionar el nº de empleado, salario, comisión, nº de departamento y fecha de la tabla EMP.
```

```
consultar("SELECT ENO, SAL, COMM, DEPTNO, HIREDATE FROM EMP")
```

[44] ✓ 0.0s

```
... (7369, 800.0, None, 20, datetime.date(1980, 12, 17))
(7499, 1600.0, 300.0, 30, datetime.date(1981, 2, 20))
(7521, 1250.0, 500.0, 30, datetime.date(1981, 2, 22))
(7566, 2975.0, None, 20, datetime.date(1981, 4, 2))
(7654, 1250.0, 1400.0, 30, datetime.date(1981, 9, 28))
(7698, 2850.0, None, 30, datetime.date(1981, 5, 1))
(7782, 2450.0, None, 10, datetime.date(1981, 6, 9))
(7788, 3000.0, None, 20, datetime.date(1987, 4, 19))
(7839, 5000.0, None, 10, datetime.date(1981, 11, 17))
(7844, 1500.0, 0.0, 30, datetime.date(1981, 9, 8))
(7876, 1100.0, None, 20, datetime.date(1987, 5, 23))
(7900, 950.0, None, 30, datetime.date(1981, 12, 3))
(7902, 3000.0, None, 20, datetime.date(1981, 12, 3))
(7934, 1300.0, None, 10, datetime.date(1982, 1, 23))
```

2. Seleccionar todas las columnas de la tabla DEPT.

```
2. Seleccionar todas las columnas de la tabla DEPT.
```

```
consultar("SELECT * FROM DEPT")
```

[45] ✓ 0.0s

```
... (10, 'ACCOUNTING', 'NEW YORK')
(20, 'RESEARCH', 'DALLAS')
(30, 'SALES', 'CHICAGO')
(40, 'OPERATIONS', 'BOSTON')
```

3. Seleccionar los nombres y los empleos de todos los empleados, ordenados por empleo.

```
3. Seleccionar los nombres y los empleos de todos los empleados, ordenados por empleo.
```

```
consultar("SELECT ENAME, JOB FROM EMP ORDER BY JOB")
```

[46] ✓ 0.1s

```
... ('SCOTT', 'ANALYST')
    ('FORD', 'ANALYST')
    ('SMITH', 'CLERK')
    ('ADAMS', 'CLERK')
    ('JAMES', 'CLERK')
    ('MILLER', 'CLERK')
    ('JONES', 'MANAGER')
    ('BLAKE', 'MANAGER')
    ('CLARK', 'MANAGER')
    ('KING', 'PRESIDENT')
    ('ALLEN', 'SALESMAN')
    ('WARD', 'SALESMAN')
    ('MARTIN', 'SALESMAN')
    ('TURNER', 'SALESMAN')
```

4. Seleccionar los empleos que hay en cada departamento, ordenados por departamento.

```
4. Seleccionar los empleos que hay en cada departamento, ordenados por departamento.
```

```
consultar("SELECT DEPTNO, JOB FROM EMP ORDER BY DEPTNO")
```

[47] ✓ 0.0s

```
... (10, 'MANAGER')
    (10, 'PRESIDENT')
    (10, 'CLERK')
    (20, 'CLERK')
    (20, 'MANAGER')
    (20, 'ANALYST')
    (20, 'CLERK')
    (20, 'ANALYST')
    (30, 'SALESMAN')
    (30, 'SALESMAN')
    (30, 'SALESMAN')
    (30, 'MANAGER')
    (30, 'SALESMAN')
    (30, 'CLERK')
```

5. Seleccionar los distintos departamentos que existen en la tabla EMP.

```
5. Seleccionar los distintos departamentos que existen en la tabla EMP.

consultar("SELECT DISTINCT DEPTNO FROM EMP")
[48] ✓ 0.1s
.. (10,)
   (20,)
   (30,)
```

6. Calcular el salario anual a percibir por cada empleado.

```
6. Calcular el salario anual a percibir por cada empleado.

consultar("SELECT ENAME, SAL*12 AS SALARIO_ANUAL FROM EMP")
[49] ✓ 0.0s
... ('SMITH', 9600.0)
    ('ALLEN', 19200.0)
    ('WARD', 15000.0)
    ('JONES', 35700.0)
    ('MARTIN', 15000.0)
    ('BLAKE', 34200.0)
    ('CLARK', 29400.0)
    ('SCOTT', 36000.0)
    ('KING', 60000.0)
    ('TURNER', 18000.0)
    ('ADAMS', 13200.0)
    ('JAMES', 11400.0)
    ('FORD', 36000.0)
    ('MILLER', 15600.0)
```

7. Mostrar el nombre del empleado y una columna que contenga el salario multiplicado por la comisión cuya cabecera sea “BONO”.

```
7. Mostrar el empleado y una columna que contenga el salario multiplicado por la comisión cuya cabecera sea "BONO"
```

```
consultar("SELECT ENAME, SAL*IFNULL(COMM,0) AS BONO FROM EMP")
```

[50] ✓ 0.0s

```
... ('SMITH', 0.0)
    ('ALLEN', 480000.0)
    ('WARD', 625000.0)
    ('JONES', 0.0)
    ('MARTIN', 1750000.0)
    ('BLAKE', 0.0)
    ('CLARK', 0.0)
    ('SCOTT', 0.0)
    ('KING', 0.0)
    ('TURNER', 0.0)
    ('ADAMS', 0.0)
    ('JAMES', 0.0)
    ('FORD', 0.0)
    ('MILLER', 0.0)
```

8. Seleccionar aquellos empleados que sean "SALESMAN".

```
8. Seleccionar aquellos empleados que sean "SALESMAN".
```

```
consultar("SELECT * FROM EMP WHERE JOB='SALESMAN'")
```

[51] ✓ 0.0s

```
... (7499, 'ALLEN', 'SALESMAN', 7698, datetime.date(1981, 2, 20), 1600.0, 300.0, 30)
    (7521, 'WARD', 'SALESMAN', 7698, datetime.date(1981, 2, 22), 1250.0, 500.0, 30)
    (7654, 'MARTIN', 'SALESMAN', 7698, datetime.date(1981, 9, 28), 1250.0, 1400.0, 30)
    (7844, 'TURNER', 'SALESMAN', 7698, datetime.date(1981, 9, 8), 1500.0, 0.0, 30)
```

9. Seleccionar aquellos empleados que no trabajen en el departamento 30.

```
9. Seleccionar aquellos empleados que no trabajen en el departamento 30.
```

```
consultar("SELECT * FROM EMP WHERE DEPTNO != 30")
```

[52] ✓ 0.1s

```
... (7782, 'CLARK', 'MANAGER', 7839, datetime.date(1981, 6, 9), 2450.0, None, 10)
    (7839, 'KING', 'PRESIDENT', None, datetime.date(1981, 11, 17), 5000.0, None, 10)
    (7934, 'MILLER', 'CLERK', 7782, datetime.date(1982, 1, 23), 1300.0, None, 10)
    (7369, 'SMITH', 'CLERK', 7902, datetime.date(1980, 12, 17), 800.0, None, 20)
    (7566, 'JONES', 'MANAGER', 7839, datetime.date(1981, 4, 2), 2975.0, None, 20)
    (7788, 'SCOTT', 'ANALYST', 7566, datetime.date(1987, 4, 19), 3000.0, None, 20)
    (7876, 'ADAMS', 'CLERK', 7788, datetime.date(1987, 5, 23), 1100.0, None, 20)
    (7902, 'FORD', 'ANALYST', 7566, datetime.date(1981, 12, 3), 3000.0, None, 20)
```

10. Seleccionar el nombre de aquellos empleados que ganen más de 2000.

```
10. Seleccionar el nombre de aquellos empleados que ganen más de 2000.

consultar("SELECT ENAME FROM EMP WHERE SAL > 2000")
[53] ✓ 0.0s
... ('JONES',)
    ('BLAKE',)
    ('CLARK',)
    ('SCOTT',)
    ('KING',)
    ('FORD',)
```

11. Seleccionar aquellos empleados que hayan entrado antes del 1/1/82

```
11. Seleccionar aquellos empleados que hayan entrado antes del 1/1/82

consultar("SELECT * FROM EMP WHERE HIREDATE < '1982-01-01'")
[54] ✓ 0.0s
... (7369, 'SMITH', 'CLERK', 7902, datetime.date(1980, 12, 17), 800.0, None, 20)
    (7499, 'ALLEN', 'SALESMAN', 7698, datetime.date(1981, 2, 20), 1600.0, 300.0, 30)
    (7521, 'WARD', 'SALESMAN', 7698, datetime.date(1981, 2, 22), 1250.0, 500.0, 30)
    (7566, 'JONES', 'MANAGER', 7839, datetime.date(1981, 4, 2), 2975.0, None, 20)
    (7654, 'MARTIN', 'SALESMAN', 7698, datetime.date(1981, 9, 28), 1250.0, 1400.0, 30)
    (7698, 'BLAKE', 'MANAGER', 7839, datetime.date(1981, 5, 1), 2850.0, None, 30)
    (7782, 'CLARK', 'MANAGER', 7839, datetime.date(1981, 6, 9), 2450.0, None, 10)
    (7839, 'KING', 'PRESIDENT', None, datetime.date(1981, 11, 17), 5000.0, None, 10)
    (7844, 'TURNER', 'SALESMAN', 7698, datetime.date(1981, 9, 8), 1500.0, 0.0, 30)
    (7900, 'JAMES', 'CLERK', 7698, datetime.date(1981, 12, 3), 950.0, None, 30)
    (7902, 'FORD', 'ANALYST', 7566, datetime.date(1981, 12, 3), 3000.0, None, 20)
```

12. Mostrar el nombre del empleado y su fecha de alta en la empresa de los empleados que son “ANALISTA”.

```
12. Mostrar el nombre del empleado y su fecha de alta en la empresa de los empleados que son “ANALISTA”.

consultar("SELECT ENAME, HIREDATE FROM EMP WHERE JOB='ANALYST'")
[55] ✓ 0.1s
... ('SCOTT', datetime.date(1987, 4, 19))
    ('FORD', datetime.date(1981, 12, 3))
```

13. Seleccionar los empleados cuyo salario sea superior al de "ADAMS".

```
13. Seleccionar los empleados cuyo salario sea superior al de "ADAMS".

consultar("""
SELECT * FROM EMP
WHERE SAL > (SELECT SAL FROM EMP WHERE ENAME='ADAMS')
""")
```

[56] ✓ 0.1s

...

(7499, 'ALLEN', 'SALESMAN', 7698, datetime.date(1981, 2, 20), 1600.0, 300.0, 30)
(7521, 'WARD', 'SALESMAN', 7698, datetime.date(1981, 2, 22), 1250.0, 500.0, 30)
(7566, 'JONES', 'MANAGER', 7839, datetime.date(1981, 4, 2), 2975.0, None, 20)
(7654, 'MARTIN', 'SALESMAN', 7698, datetime.date(1981, 9, 28), 1250.0, 1400.0, 30)
(7698, 'BLAKE', 'MANAGER', 7839, datetime.date(1981, 5, 1), 2850.0, None, 30)
(7782, 'CLARK', 'MANAGER', 7839, datetime.date(1981, 6, 9), 2450.0, None, 10)
(7788, 'SCOTT', 'ANALYST', 7566, datetime.date(1987, 4, 19), 3000.0, None, 20)
(7839, 'KING', 'PRESIDENT', None, datetime.date(1981, 11, 17), 5000.0, None, 10)
(7844, 'TURNER', 'SALESMAN', 7698, datetime.date(1981, 9, 8), 1500.0, 0.0, 30)
(7902, 'FORD', 'ANALYST', 7566, datetime.date(1981, 12, 3), 3000.0, None, 20)
(7934, 'MILLER', 'CLERK', 7782, datetime.date(1982, 1, 23), 1300.0, None, 10)

14. Seleccionar los empleados que trabajan en el mismo departamento que "CLARK".

```
14. Seleccionar los empleados que trabajan en el mismo departamento que "CLARK".

consultar("""
SELECT * FROM EMP
WHERE DEPTNO = (SELECT DEPTNO FROM EMP WHERE ENAME='CLARK')
""")
```

[57] ✓ 0.0s

...

(7782, 'CLARK', 'MANAGER', 7839, datetime.date(1981, 6, 9), 2450.0, None, 10)
(7839, 'KING', 'PRESIDENT', None, datetime.date(1981, 11, 17), 5000.0, None, 10)
(7934, 'MILLER', 'CLERK', 7782, datetime.date(1982, 1, 23), 1300.0, None, 10)

15. Encontrar a los empleados cuyo jefe es "BLAKE".

```
15. Encontrar a los empleados cuyo jefe es "BLAKE".

consultar("""
SELECT * FROM EMP
WHERE MGR = (SELECT ENO FROM EMP WHERE ENAME='BLAKE')
""")
[58] ✓ 0.0s

... (7499, 'ALLEN', 'SALESMAN', 7698, datetime.date(1981, 2, 20), 1600.0, 300.0, 30)
(7521, 'WARD', 'SALESMAN', 7698, datetime.date(1981, 2, 22), 1250.0, 500.0, 30)
(7654, 'MARTIN', 'SALESMAN', 7698, datetime.date(1981, 9, 28), 1250.0, 1400.0, 30)
(7844, 'TURNER', 'SALESMAN', 7698, datetime.date(1981, 9, 8), 1500.0, 0.0, 30)
(7900, 'JAMES', 'CLERK', 7698, datetime.date(1981, 12, 3), 950.0, None, 30)
```

16. Seleccionar el nombre de los vendedores que ganen más de 1500.

```
16. Seleccionar el nombre de los vendedores que ganen más de 1500.

consultar("""
SELECT ENAME FROM EMP
WHERE JOB='SALESMAN' AND SAL > 1500
""")
[59] ✓ 0.1s

... ('ALLEN',)
```

17. Seleccionar aquellos empleados que tienen comisión.

```
17. Seleccionar aquellos empleados que tienen comisión.

consultar("SELECT * FROM EMP WHERE COMM IS NOT NULL")
[60] ✓ 0.0s

... (7499, 'ALLEN', 'SALESMAN', 7698, datetime.date(1981, 2, 20), 1600.0, 300.0, 30)
(7521, 'WARD', 'SALESMAN', 7698, datetime.date(1981, 2, 22), 1250.0, 500.0, 30)
(7654, 'MARTIN', 'SALESMAN', 7698, datetime.date(1981, 9, 28), 1250.0, 1400.0, 30)
(7844, 'TURNER', 'SALESMAN', 7698, datetime.date(1981, 9, 8), 1500.0, 0.0, 30)
```

18. Seleccionar aquellos que se llamen "SMITH", "ALLEN" o "SCOTT".

```
18. Seleccionar aquellos que se llamen "SMITH", "ALLEN" o "SCOTT".

consultar("""
SELECT * FROM EMP
WHERE ENAME IN ('SMITH','ALLEN','SCOTT')
""")
[61] ✓ 0.0s

... (7369, 'SMITH', 'CLERK', 7902, datetime.date(1980, 12, 17), 800.0, None, 20)
(7499, 'ALLEN', 'SALESMAN', 7698, datetime.date(1981, 2, 20), 1600.0, 300.0, 30)
(7788, 'SCOTT', 'ANALYST', 7566, datetime.date(1987, 4, 19), 3000.0, None, 20)
```

19. Seleccionar aquellos que no se llamen "SMITH", "ALLEN" o "SCOTT".

```
19. Seleccionar aquellos que no se llamen "SMITH", "ALLEN" o "SCOTT".

consultar("""
SELECT * FROM EMP
WHERE ENAME NOT IN ('SMITH','ALLEN','SCOTT')
""")
[62] ✓ 0.0s

... (7521, 'WARD', 'SALESMAN', 7698, datetime.date(1981, 2, 22), 1250.0, 500.0, 30)
(7566, 'JONES', 'MANAGER', 7839, datetime.date(1981, 4, 2), 2975.0, None, 20)
(7654, 'MARTIN', 'SALESMAN', 7698, datetime.date(1981, 9, 28), 1250.0, 1400.0, 30)
(7698, 'BLAKE', 'MANAGER', 7839, datetime.date(1981, 5, 1), 2850.0, None, 30)
(7782, 'CLARK', 'MANAGER', 7839, datetime.date(1981, 6, 9), 2450.0, None, 10)
(7839, 'KING', 'PRESIDENT', None, datetime.date(1981, 11, 17), 5000.0, None, 10)
(7844, 'TURNER', 'SALESMAN', 7698, datetime.date(1981, 9, 8), 1500.0, 0.0, 30)
(7876, 'ADAMS', 'CLERK', 7788, datetime.date(1987, 5, 23), 1100.0, None, 20)
(7900, 'JAMES', 'CLERK', 7698, datetime.date(1981, 12, 3), 950.0, None, 30)
(7902, 'FORD', 'ANALYST', 7566, datetime.date(1981, 12, 3), 3000.0, None, 20)
(7934, 'MILLER', 'CLERK', 7782, datetime.date(1982, 1, 23), 1300.0, None, 10)
```

20. Seleccionar los empleados que trabajen en "CHICAGO".

```
20. Seleccionar los empleados que trabajen en "CHICAGO".

consultar("""
SELECT E.*
FROM EMP E
JOIN DEPT D ON E.DEPTNO = D.DEPTNO
WHERE D.LOC='CHICAGO'
""")
[63] ✓ 0.1s
... (7499, 'ALLEN', 'SALESMAN', 7698, datetime.date(1981, 2, 20), 1600.0, 300.0, 30)
(7521, 'WARD', 'SALESMAN', 7698, datetime.date(1981, 2, 22), 1250.0, 500.0, 30)
(7654, 'MARTIN', 'SALESMAN', 7698, datetime.date(1981, 9, 28), 1250.0, 1400.0, 30)
(7698, 'BLAKE', 'MANAGER', 7839, datetime.date(1981, 5, 1), 2850.0, None, 30)
(7844, 'TURNER', 'SALESMAN', 7698, datetime.date(1981, 9, 8), 1500.0, 0.0, 30)
(7900, 'JAMES', 'CLERK', 7698, datetime.date(1981, 12, 3), 950.0, None, 30)
```

21. Seleccionar aquellos empleados que trabajen en el departamento 10 o en el 20.

```
21. Seleccionar aquellos empleados que trabajen en el departamento 10 o en el 20.

consultar("SELECT * FROM EMP WHERE DEPTNO IN (10,20)")
[64] ✓ 0.1s
... (7782, 'CLARK', 'MANAGER', 7839, datetime.date(1981, 6, 9), 2450.0, None, 10)
(7839, 'KING', 'PRESIDENT', None, datetime.date(1981, 11, 17), 5000.0, None, 10)
(7934, 'MILLER', 'CLERK', 7782, datetime.date(1982, 1, 23), 1300.0, None, 10)
(7369, 'SMITH', 'CLERK', 7902, datetime.date(1980, 12, 17), 800.0, None, 20)
(7566, 'JONES', 'MANAGER', 7839, datetime.date(1981, 4, 2), 2975.0, None, 20)
(7788, 'SCOTT', 'ANALYST', 7566, datetime.date(1987, 4, 19), 3000.0, None, 20)
(7876, 'ADAMS', 'CLERK', 7788, datetime.date(1987, 5, 23), 1100.0, None, 20)
(7902, 'FORD', 'ANALYST', 7566, datetime.date(1981, 12, 3), 3000.0, None, 20)
```